

Capítulo 22

Templates

Exercícios de Autorrevisão — Resolução Didática

“Templates são o polimorfismo estático de C++: permitem escrever código genérico que funciona para vários tipos sem custo em tempo de execução. Diferente do polimorfismo dinâmico (Cap. 21) — onde a vtable resolve a chamada em tempo de execução via herança —, templates são resolvidos pelo compilador, gerando código especializado para cada tipo usado. Esse capítulo introduz dois sabores: **template** de função (funções parametrizadas por tipo) e **template** de classe (classes parametrizadas por tipo e/ou valores).”

1 Exercício 22.1 — Verdadeiro ou Falso

(a) Parâmetros de template podem ser usados para tipos de argumento, tipo de retorno e variáveis locais

Resposta: Verdadeiro.

Dentro do corpo de um template, o parâmetro de tipo T é tratado como qualquer outro tipo:

```
template <typename T>
T media(T a, T b) {           // T como tipo de retorno e de argumento
    T soma = a + b;           // T como tipo de variavel local
    return soma / 2;
}
```

(b) typename e class em parâmetro de template significam “qualquer tipo de classe”

Resposta: Falso.

Correção: typename e class (intercambiáveis nesse contexto) representam *qualquer tipo*, incluindo tipos primitivos como int, double, char, bool, e também ponteiros, referências e tipos de classe. Não se limitam a classes definidas pelo usuário.

```
selectionSort<int>(arr, 10);    // T = int (primitivo, OK)
selectionSort<double>(arr, 10); // T = double (primitivo, OK)
selectionSort<Funcionario>(arr, 10); // T = Funcionario (classe, tambem OK)
```

(c) Um template de função pode ser sobrecarregado por outro com o mesmo nome**Resposta: Verdadeiro.**

Templates seguem as regras normais de sobrecarga — desde que difiram em *número ou tipos* de parâmetros:

```
template <typename T>
void printArray(const T arr[], int tamanho);

template <typename T>
int printArray(const T arr[], int tamanho,
               int low, int high);    // sobrecarga: 2 args extras

void printArray(const char *const arr[], int tamanho); // tambem
```

Esse é exatamente o cenário do Exercício 22.4.

(d) Nomes de parâmetros de template entre templates precisam ser únicos**Resposta: Falso.**

Correção: cada template tem seu próprio *escopo* de parâmetros, então pode-se reusar nomes:

```
template <typename T> T menor(T a, T b);           // T local a esta
// funcao
template <typename T> T maior(T a, T b);          // outro T, ok!
template <typename T> class Pilha { /* outro T */ };
```

(e) Toda função-membro fora de um template de classe precisa começar com cabeçalho template**Resposta: Verdadeiro.**

Quando uma função-membro é definida *fora* da classe template, ela também é um template:

```
template <typename T> class Array {
public:
    Array(int s);
    // ...
};

// Definicao da funcao-membro fora da classe:
template <typename T>           // <-- cabeçalho template OBRIGATORIO
Array<T>::Array(int s) {
    // ...
}
```

(f) Uma função friend de um template de classe precisa ser uma especialização de template**Resposta: Falso.****Correção:** uma `friend` de um template de classe pode ser:

- Uma função não-template;
- Uma especialização de template de função;
- Outra classe;
- Outra especialização de template de classe.

Ou seja, não há obrigação de que seja template. Veremos exemplos no Exercício 22.19.

(g) Templates de classe com `static` compartilham uma só cópia desse `static`**Resposta: Falso.****Correção:** cada *especialização* do template de classe é um *tipo distinto* para todos os efeitos do compilador. Logo, cada uma tem sua *própria* cópia de `static`.

```
template <typename T> class Contador {
public:
    static int n;
};
template <typename T> int Contador<T>::n = 0;

Contador<int>::n    = 5;
Contador<float>::n = 10;
// Contador<int>::n    continua = 5    (cada um tem sua copia!)
// Contador<float>::n continua = 10
```

Este é o tema do Exercício 22.20.

2 Exercício 22.2 — Vocabulário*Preencha os espaços.***(a) Templates geram _____ (de função) e _____ (de classe)****Resposta:** especializações de template de função; especializações de template de classe.Quando você escreve `selectionSort<int>(arr, n)`, o compilador *gera* uma função concreta de `int` a partir do template. Essa função é uma *especialização*. O mesmo vale para `Array<float, 5>` — é uma especialização do template `Array`.**Observação de Engenharia de Software**

A nomenclatura aqui pode confundir: “especialização” tem dois significados sobrepostos em C++:

- *Especialização implícita* (gerada pelo compilador): `Array<int, 5>` é uma especialização implícita.

- *Especialização explícita* (escrita pelo programador): `template<> class Array<float>` é uma especialização explícita.

O Exercício 22.7 contém exemplos das duas formas.

(b) Templates começam com _____ seguido de lista entre _____

Resposta: `template`; sinais `<` e `>`.

```
template <typename T, int N>
class Array { /* ... */ };
//~~~~~
//  cabeçalho do template, com parametros entre < >
```

(c) Funções geradas por template têm o mesmo nome; o compilador usa resolução de _____ para chamar

Resposta: sobrecarga.

`selectionSort(vi, 5)` (com `vi` sendo `int[]`) e `selectionSort(vf, 3)` (com `vf` sendo `float[]`) chamam funções *distintas* (especializações distintas). O compilador resolve qual chamar pelas regras de sobrecarga.

(d) Templates de classe também são chamados de tipos _____

Resposta: parametrizados.

Um “tipo parametrizado” é um tipo que aceita parâmetros (de tipo ou de valor). `Array<int, 5>` é o tipo parametrizado por `int` e `5`. Esse é o tema do Exercício 22.14.

(e) Operador para unir definição de função-membro ao escopo do template de classe

Resposta: `::` (operador binário de resolução de escopo).

```
template <typename T>
Array<T>::Array(int s) { /* ... */ }
//      ~~ <-- operador ::
```

(f) Dados-membro `static` de especializações de template devem ser definidos em escopo de _____

Resposta: namespace global.

Igual a qualquer `static` de classe, mas com o cabeçalho `template`:

```
// No header da classe:
template <typename T> class Pilha {
public:
```

```
    static int contador;  
};  
  
// No .cpp ou no header (definição no namespace global):  
template <typename T> int Pilha<T>::contador = 0;
```

Síntese conceitual

A autorrevisão estabeleceu os mecanismos centrais de templates em C++:

- **Sintaxe:** `template <typename T, ...>` introduz parâmetros.
 - **Generalidade:** `typename/class` aceitam qualquer tipo (primitivos, ponteiros, classes).
 - **Sobrecarga:** templates de função podem coexistir entre si e com funções não-template.
 - **Especializações:** cada uso com tipos concretos gera uma especialização distinta, com `static` próprio.
 - **Consequência:** um template é um *molde*; especializações são *cópias concretas* desse molde.
-